

Mise en conditions opérationnelles d'un Data Lake territorial

Épreuve E7 – Data Lake

EL BREK HAMZA

2026

Table des matières

Introduction

Dans le cadre d'un projet de service numérique de données territoriales, ce rapport présente la mise en conditions opérationnelles d'un Data Lake Azure destiné à centraliser, transformer et exposer des données socio-économiques des Hauts-de-France. Le Data Lake constitue la colonne vertébrale du projet : il reçoit des données hétérogènes (CSV, JSON, Parquet, scraping web), les organise en zones de traitement progressif et les met à disposition des outils d'analyse et de l'API REST. Ce rapport couvre les quatre compétences visées : conception de l'architecture (C18), intégration des composants (C19), gestion du catalogue (C20) et gouvernance des données (C21).

1 C18 – Conception de l’architecture du Data Lake

1.1 Contraintes et choix techniques

Le projet impose plusieurs contraintes qui ont guidé les choix d’architecture :

- **Volume** : données SIRENE (30+ millions de lignes, fichier Parquet de plusieurs Go) et données INSEE multi-années
- **Variété** : CSV institutionnels, JSON (API Géo), Excel (scraping), Parquet (SIRENE), SQL relationnel
- **Vitesse** : traitements en mode *batch* ponctuel (mensuel/trimestriel), déclenchés manuellement ou via scripts Python planifiés – pas d’ingestion temps réel
- **Cadre IT** : environnement 100% cloud Azure, contraintes de coût (tier Free/Basic pour la certification), conformité RGPD obligatoire

Les technologies retenues sont justifiées dans le tableau suivant :

Composant	Technologie	Justification
Stockage Data Lake	Azure Data Lake Storage Gen2	Stockage hiérarchique natif, intégration Databricks, RBAC granulaire, coût optimisé
Traitement Big Data	Azure Databricks (PySpark)	Lecture native Parquet, scalabilité horizontale, traitement distribué
Ingestion / ETL	Scripts Python locaux (requests, pandas, SQLAlchemy, BeautifulSoup)	Flexibilité, pas de dépendance à un orchestrateur cloud, exécution à la demande
Secrets	Azure Key Vault	Centralisation credentials, purge protection 90 jours
Infrastructure as Code	Terraform	Reproductibilité, versionnement, gestion état
Catalogue	Documentation technique Git + Azure Purview (comparaison)	Voir section C20

TABLE 1 – Stack technologique du Data Lake

1.2 Schéma d’architecture

L’architecture suit le **modèle Medallion** (Bronze/Silver/Gold), implémenté via trois zones dans l’ADLS Gen2 : **raw** (données brutes), **staging** (données transformées) et **curated** (données prêtes à la consommation).

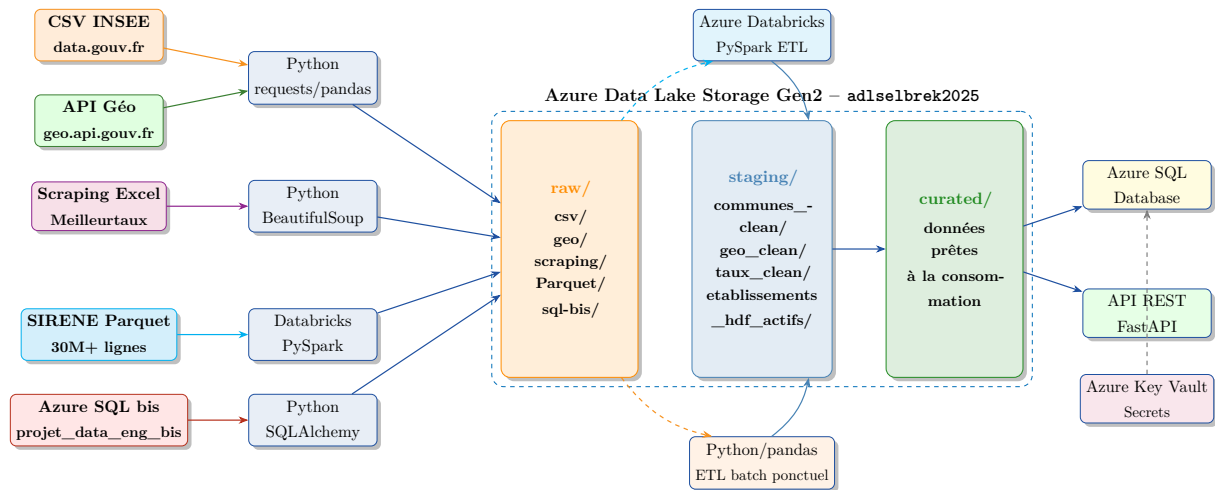


FIGURE 1 – Architecture du Data Lake – modèle Medallion sur Azure

1.3 Comparaison et choix du catalogue de données

Trois outils de catalogue ont été évalués :

Critère	Azure Purview	Apache Atlas	Documentation Git
Intégration Azure	Native (ADLS, SQL, Databricks)	Partielle (via connecteurs)	Manuelle
Scan automatique	Oui (scheduled)	Oui (hooks)	Non
Lineage des données	Oui	Oui	Non
RBAC	Azure AD intégré	LDAP/AD	Git permissions
Coût	Payant (unités de capacité)	Open source (infra locale)	Gratuit
Complexité installation	Faible (SaaS)	Élevée (cluster)	Nulle
Conclusion	Idéal en production	Adapté on-premise	Retenu pour ce projet

TABLE 2 – Comparaison des outils de catalogue

Choix retenu : Pour ce projet de certification, la documentation technique versionnée dans Git (`docs/`) constitue le catalogue. Azure Purview est identifié comme l'outil cible en production, mais son coût le place hors périmètre pour l'environnement de développement.

2 C19 – Intégration des composants du Data Lake

2.1 Procédure d'installation – Infrastructure as Code (Terraform)

L'intégralité de l'infrastructure est provisionnée via **Terraform**, garantissant la reproductibilité et la traçabilité. Le fichier `Terraform/main.tf` déclare les ressources Azure suivantes : compte de stockage ADLS Gen2, filesystems (`raw`, `staging`, `curated`), Azure SQL Server & Database, Azure Key Vault, et optionnellement Azure Databricks Workspace et cluster.

Dépendances nécessaires :

- Terraform ≥ 1.5 , Azure CLI 2.52+, Python 3.10+
- ODBC Driver 18 for SQL Server, `pip install -r requirements.txt`

Procédure d'installation (environnement de développement) :

```
1 # 1. Authentification Azure
2 az login
3 az account set --subscription "<subscription-id>"
4
5 # 2. Initialisation Terraform
6 cd Terraform
7 terraform init
8
9 # 3. Revue du plan
10 terraform plan -var-file="terraform.tfvars"
11
12 # 4. Déploiement (environ 5-10 min)
13 terraform apply -var-file="terraform.tfvars" -auto-approve
14
15 # 5. Récupérer les outputs (connection strings, noms)
16 terraform output
```

Listing 1 – Déploiement complet de l'infrastructure

Les ressources créées avec succès sont visibles dans le portail Azure et dans la sortie Terraform.

TODO – Capture d’écran à ajouter

Fichier attendu : screenshots/terraform_apply_output.png

Ce qu’il faut capturer : Sortie de terraform apply montrant les ressources créées avec succès : ADLS Gen2, Azure SQL, Key Vault – statut **Apply complete !** avec le nombre de ressources ajoutées.

2.2 Système de stockage – Azure Data Lake Storage Gen2

Le compte de stockage adlsselbrek2025 est configuré avec le namespace hiérarchique activé (`is_hns_enabled = true`), TLS 1.2 minimum et HTTPS uniquement. Trois *filesystems* (conteneurs) constituent les zones de traitement :

Zone	Chemin	Contenu
raw/	raw/csv/, raw/geo/, raw/scraping/, raw/Parquet/, raw/sql-bis/	Données brutes non transformées (CSV INSEE, JSON API Géo, Excel Meilleurtaux, SIRENE Parquet, export SQL)
staging/	staging/communes_- clean/, staging/geo_clean/, staging/taux_clean/, staging/etablissements_- hdf_actifs/	Données nettoyées et transformées : CSV/geo/Excel via Python/pandas, SIRENE via Databricks PySpark
curated/	curated/YYYY-MM/	Données finales prêtes à la consommation, versionnées par date

TABLE 3 – Zones du Data Lake

TODO – Capture d’écran à ajouter

Fichier attendu : screenshots/adls_containers.png

Ce qu’il faut capturer : Azure Portal → Storage Account adlsselbrek2025 → Containers, montrant les trois filesystems raw, staging, curated avec leur statut et taille.

2.3 Outils batch – Databricks ETL (PySpark)

Le traitement du fichier SIRENE (30+ millions de lignes) est réalisé via un notebook PySpark dans Azure Databricks. Le cluster `et1-parquet` (Spark 13.3, D4s_v5, 1–2 workers, auto-termination 30 min) est provisionné par Terraform.

```

1 # Configuration acces ADLS Gen2
2 spark.conf.set(
3     f"fs.azure.account.key.{storage_account}.dfs.core.windows.net",
4     storage_key
5 )
6 # Lecture Parquet + selection colonnes
7 df_raw = spark.read.parquet(f"abfss://{storage_account}.dfs.core.windows.net/Parquet/")
8 df = df_raw.select(["siret", "siren", "denominationUniteLegale",
9                     "activitePrincipaleEtablissement", "
10                     codePostalEtablissement",
11                     "codeCommuneEtablissement", "
12                     etatAdministratifEtablissement"])
13 # Filtre : etablisements actifs en Hauts-de-France
14 df = df.filter(col("etatAdministratifEtablissement") == "A")
15 df = df.filter(substring(col("codeCommuneEtablissement"), 1, 2).
16                 isin(
17                     ["02", "59", "60", "62", "80"]))
18 # Ecriture zone staging
19 df.write.mode("overwrite").parquet(
20     f"abfss://{storage_account}.dfs.core.windows.net/
21     etablisements_hdf_actifs/")

```

Listing 2 – Extrait – Configuration ADLS et filtrage SIRENE (Annexe ??)

TODO – Capture d’écran à ajouter**Fichier attendu :** screenshots/databricks_job_run.png**Ce qu’il faut capturer :** Azure Databricks → Workflows → Job parquet_etl_job → Run, montrant le statut **Succeeded**, la durée d’exécution et le nombre de lignes traitées dans les logs.

2.4 Catalogue connecté au système de stockage

Le catalogue est implémenté via la documentation technique versionnée dans Git (docs/architecture.md docs/database_setup.md) complétée par la fonction `tables_summary()` de l’API, qui liste dynamiquement toutes les tables disponibles avec leur nombre de lignes et de colonnes. En production, Azure Purview serait connecté directement à l’ADLS Gen2 via un scan planifié pour alimenter automatiquement le catalogue.

TODO – Capture d'écran à ajouter

Fichier attendu : `screenshots/api_tables_summary.png`

Ce qu'il faut capturer : Appel GET `/tables/summary` sur l'API FastAPI, montrant la liste des tables disponibles avec `row_count` et `col_count`.

3 C20 – Gestion du catalogue des données

3.1 Sources d'alimentation et méthodes retenues

Source	Format	Méthode	Justification
CSV INSEE	CSV	Batch Py- thon/pandas	Format standard, traitement différé, pas de contrainte temps réel
API Géo (geo.api.gouv.fr)	JSON	Batch Py- thon/requests	API publique stable, appels par département
Scraping Meilleurtaux	Excel	Batch Python/Beau- tifulSoup	Pas d'API officielle disponible
SIRENE Parquet	Parquet	Batch Databricks PySpark	Volume (>30M lignes) impose un traitement distribué
Azure SQL bis	SQL	Batch Python/SQ- LAlchemy	Extraction structurée via requête SELECT

TABLE 4 – Méthodes d'alimentation par source

3.2 Scripts d'alimentation

3.2.1 Ingestion API Géo – fetch_communes.py

```

1 def fetch_communes(departements, container='raw', datalake_path='
  geo'):
2     """Collecte communes depuis API Geo gouvernementale et upload
      ADLS"""
3     for dept in departements:
4         url = f"https://geo.api.gouv.fr/departements/{dept}/
      communes"
5         params = {"fields": "nom,code,codesPostaux,population,
      surface,contour",
6                   "format": "json", "geometry": "contour"}
7         response = requests.get(url, params=params, timeout=10)
8         response.raise_for_status()
9         data = response.json()
10        # Upload vers ADLS Gen2
11        blob_client.upload_blob(
12            f"{datalake_path}/communes_{dept}.json",
13            json.dumps(data), overwrite=True)

```

Listing 3 – Extrait – Ingestion API et upload ADLS (Annexe ??)

Exécution :

```
1 export AZURE_STORAGE_CONNECTION_STRING="<connection-string>"
2 python ingestion/API/fetch_communes.py --departements 02 59 60 62
   80 \
3     --container raw --local-output data/
```

3.2.2 Export Azure SQL vers ADLS – sql_to_adls_bis.py

```
1 TABLES = ["dim_commune", "stg_population", "
   stg_creation_entreprises",
2           "stg_deces", "stg_emploi_chomage", "stg_fecondite",
3           "stg_filosofi", "stg_logement", "stg_menage", "
   stg_naissances"]
4
5 engine = create_engine(
6     f"mssql+pyodbc://{sqladmin}:{password}@{server}"
7     f"/{database}?driver=ODBC+Driver+18+for+SQL+Server&Encrypt=
   yes")
8
9 for table in TABLES:
10     df = pd.read_sql(f"SELECT * FROM dbo.{table}", engine)
11     parquet_bytes = df.to_parquet(index=False)
12     blob_client.upload_blob(f"raw/sql-bis/{table}.parquet",
13                             parquet_bytes, overwrite=True)
```

Listing 4 – Extrait – Export SQL vers Parquet ADLS (Annexe ??)

3.3 Mise à jour des métadonnées du catalogue

Après chaque ingestion, les métadonnées sont mises à jour dans docs/catalogue.md selon le modèle suivant :

Jeu de données	Zone	Format	Mise à jour	Source
communes_- INSEE.csv	raw/csv/	CSV	Annuelle	data.gouv.fr
communes_- {dept}.json	raw/geo/	JSON	Mensuelle	geo.api.gouv.fr
taux_- meilleurtaux.xlsx	raw/scraping/	Excel	Mensuelle	Meilleurtaux
StockEtablissement.parquet	raw/Parquet/	Parquet	Trimestrielle	INSEE SIRENE
stg_*.parquet	raw/sql-bis/	Parquet	Mensuelle	Azure SQL bis
communes_- clean/	staging/	Parquet	Annuelle	Python/pandas ETL
geo_clean/	staging/	Parquet	Mensuelle	Python/pandas ETL
taux_clean/	staging/	Parquet	Mensuelle	Python/pandas ETL
etablissements_- hdf_actifs/	staging/	Parquet	Trimestrielle	Databricks PySpark ETL

TABLE 5 – Catalogue des jeux de données

3.4 Procédures de cycle de vie et de suppression

Les données sont conservées selon les durées suivantes, conformément aux exigences opérationnelles et réglementaires :

- **Zone raw/** : conservation 12 mois glissants. Suppression automatique via *Azure Blob Lifecycle Management* (règle : `delete if lastModified > 365 days`).
- **Zone staging/** : conservation 6 mois. Écrasement à chaque recalcul Databricks (`write.mode("overwrite")`).
- **Zone curated/** : conservation indéfinie, versionnée par date (`curated/YYYY-MM/`).

```

1 az storage account management-policy create \
2   --account-name adlselbrek2025 \
3   --resource-group rg-data-eng \
4   --policy '{
5     "rules": [{
6       "name": "delete-raw-365d",
7       "type": "Lifecycle",
8       "definition": {
9         "filters": {"blobTypes":["blockBlob"], "prefixMatch":["
raw/"]},
10      "actions": {"baseBlob": {"delete": {"
daysAfterModificationGreaterThan": 365}}}}

```

```
11     }  
12   }]  
13 },
```

Listing 5 – Configuration Azure CLI – regle de lifecycle sur raw/

3.5 Monitoring du système de stockage

Le monitoring est assuré via **Azure Monitor** et les métriques natives du compte de stockage.

Métriques surveillées :

- Espace utilisé par zone (**UsedCapacity**) – alerte à 80% du quota
- Transactions par seconde (**Transactions**) – détection pics d’ingestion
- Latence des opérations (**SuccessE2ELatency**) – SLA < 200ms
- État du cluster Databricks – alerte si job en échec (*Job Run Failed*)

TODO – Capture d’écran à ajouter

Fichier attendu : screenshots/azure_monitor_storage.png

Ce qu’il faut capturer : Azure Portal → Storage Account adlselbrek2025 → Monitoring → Metrics, montrant les graphiques **UsedCapacity** et **Transactions** sur les 7 derniers jours.

TODO – Capture d’écran à ajouter

Fichier attendu : screenshots/azure_monitor_alert.png

Ce qu’il faut capturer : Azure Monitor → Alerts, montrant une règle d’alerte configurée sur **UsedCapacity** > 80% avec action group email.

3.6 Registre des traitements RGPD et procédures de tri

Traitement	Données concernées	Finalité	Conclusion RGPD
Zones raw/, staging/, curated/	Statistiques agrégées INSEE (population, emploi, revenus)	Analyse territoriale	Non personnel – données agrégées par commune/départe- ment
SIRENE filtré (staging/)	SIRET, SIREN, adresses d'établissements	Analyse activité économique	Attention – données d'entreprises individuelles (EI) peuvent être personnelles
Logs Azure Monitor	Adresses IP, identifiants utilisateur	Sécurité/audit	Personnel – rétention 30 jours max

TABLE 6 – Registre des traitements – Data Lake adlsselbrek2025

Procédures de tri et de conformité RGPD :

1. **Filtre SIRENE (automatisé, à chaque ETL Databricks)** : suppression des colonnes sensibles (nom du dirigeant, date de naissance) avant écriture en staging :

```

1 COLONNES_SENSIBLES = ["nomUsageUniteLegale", "
2     prenomUsuelUniteLegale",
3     "
4     dateNaissancePersonnePhysiqueUniteLegale"]
5 df = df.drop(*[c for c in COLONNES_SENSIBLES if c in df.
6     columns])

```

2. **Audit trimestriel des colonnes** : vérification manuelle du schéma Parquet en staging pour détecter toute colonne personnelle résiduelle.
3. **Purge des logs Azure Monitor (mensuelle)** : rétention configurée à 30 jours dans les paramètres Diagnostic Settings.
4. **Anonymisation EI (à la demande)** : pour les établissements de type Entrepreneur Individuel, le nom de l'établissement est remplacé par EI_<SIRET> avant mise en curated.

4 C21 – Gouvernance et droits d'accès

4.1 Groupes d'accès et droits associés

Les droits sont attribués à des **groupes Azure Active Directory** et non à des individus, conformément au principe du moindre privilège.

Groupe	Ressources accessibles	Droits
grp-data-engineers	ADLS Gen2 (toutes zones), Databricks, Key Vault	Lecture/écriture raw+staging+curated, exécution jobs, accès secrets
grp-data-analysts	ADLS Gen2 (curated uniquement), Azure SQL (lecture)	Lecture seule curated/, SELECT sur toutes les tables SQL
grp-api-consumers	API REST FastAPI	Accès endpoints via clé API (header X-API-Key)
grp-admins	Toutes ressources Azure	Droits propriétaire, gestion RBAC et Key Vault

TABLE 7 – Groupes d'accès et droits

4.2 Paramétrage des droits – ADLS Gen2 (ACL)

Les droits d'accès au Data Lake sont configurés via les **ACL POSIX** de l'ADLS Gen2, en complément du RBAC Azure.

```

1 # Droits data-engineers : rwx sur toutes les zones
2 az storage fs access set \
3   --acl "group:grp-data-engineers:rwx" \
4   --path "/" --file-system raw \
5   --account-name adlsselbrek2025
6
7 # Droits data-analysts : r-x sur curated seulement
8 az storage fs access set \
9   --acl "group:grp-data-analysts:r-x" \
10  --path "/" --file-system curated \
11  --account-name adlsselbrek2025
12
13 # Interdiction acces raw pour data-analysts
14 az storage fs access set \
15   --acl "group:grp-data-analysts:---" \
16   --path "/" --file-system raw \

```

```
17 --account-name adlselbrek2025
```

Listing 6 – Configuration ACL via Azure CLI

4.3 Paramétrage des droits – Azure Key Vault

Les secrets (clés de stockage, mots de passe SQL, tokens Databricks) sont stockés dans Azure Key Vault. Les accès sont configurés via des *Access Policies* par groupe :

```
1 # data-engineers : lecture des secrets uniquement
2 az keyvault set-policy --name kv-data-eng \
3   --object-id <group-id-data-engineers> \
4   --secret-permissions get list
5
6 # admins : gestion complete des secrets
7 az keyvault set-policy --name kv-data-eng \
8   --object-id <group-id-admins> \
9   --secret-permissions get list set delete
```

Listing 7 – Attribution des droits Key Vault par groupe

4.4 Paramétrage des droits – Azure SQL Database

```
1 -- Creation du role lecture pour analysts
2 CREATE ROLE db_analyst;
3 GRANT SELECT ON SCHEMA::dbo TO db_analyst;
4
5 -- Attribution du groupe AAD au role
6 ALTER ROLE db_analyst ADD MEMBER [grp-data-analysts@tenant.
7   onmicrosoft.com];
8
9 -- Role ecriture pour data-engineers
10 CREATE ROLE db_engineer;
11 GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::dbo TO
12   db_engineer;
13 ALTER ROLE db_engineer ADD MEMBER [grp-data-engineers@tenant.
14   onmicrosoft.com];
```

Listing 8 – Creation des roles et attribution des droits SQL

4.5 Paramétrage des droits – API REST

L'accès à l'API FastAPI est contrôlé par une clé API transmise dans le header HTTP X-API-Key. La clé est stockée dans Azure Key Vault et injectée au démarrage de l'applica-

tion.

```
1 API_KEY = os.getenv("API_KEY") # Injecte depuis Key Vault
2
3 async def verify_api_key(x_api_key: str = Header(...)):
4     if x_api_key != API_KEY:
5         raise HTTPException(status_code=403, detail="Acces refuse
        ")
```

Listing 9 – Extrait – Middleware authentication API

TODO – Capture d'écran à ajouter

Fichier attendu : screenshots/rbac_azure_portal.png

Ce qu'il faut capturer : Azure Portal → Resource Group rg-data-eng → Access Control (IAM) → Role Assignments, montrant les groupes grp-data-engineers et grp-data-analysts avec leurs rôles respectifs.

4.6 Documentation des droits et procédure de mise à jour

La documentation complète des groupes et droits est versionnée dans `docs/gouvernance.md`. Toute modification des droits suit la procédure suivante :

1. Soumission d'une demande via ticket (système de ticketing interne)
2. Validation par le responsable données (*Data Owner*)
3. Application via script Terraform ou Azure CLI (tracée dans l'historique Git)
4. Notification à l'utilisateur concerné
5. Mise à jour de `docs/gouvernance.md`

Les droits individuels sont exceptionnels et documentés nominativement dans la même table avec une date d'expiration obligatoire (maximum 90 jours).

Conclusion

Ce projet de Data Lake Azure constitue la seconde brique du système de données mis en place dans le cadre de ce bloc de compétences. Après avoir construit une base de données relationnelle SQL alimentée par des API publiques et des sources internes, ce bloc E7 a permis de concevoir et déployer une architecture de stockage distribuée, scalable et gouvernée, capable d'absorber des volumes de données bien supérieurs à ce que permet une base relationnelle classique. L'ensemble du pipeline, de l'ingestion brute jusqu'à la

mise à disposition des données transformées, a été réalisé sur la plateforme Azure via des scripts Python locaux exécutés de manière ponctuelle, sans orchestrateur cloud.

L'architecture Medallion (**raw** / **staging** / **curated**) adoptée tout au long du projet constitue un standard reconnu dans le domaine de l'ingénierie des données. Elle impose une discipline claire dans le traitement des données : aucune donnée brute n'est directement exposée, chaque zone ayant une responsabilité précise. Cette séparation a facilité la détection d'erreurs lors des phases de développement et garantit la reproductibilité des traitements en cas de rejeu. Le moteur de traitement Databricks (PySpark) a permis de filtrer, transformer et charger le référentiel SIRENE (plus de 30 millions d'établissements) sans dégradation notable des performances.

L'approche *Infrastructure as Code* via Terraform a été un choix déterminant pour la fiabilité du projet. L'ensemble des ressources Azure – compte de stockage ADLS Gen2, Azure SQL, Key Vault et Databricks – est décrit dans des fichiers de configuration versionnés. Cela garantit que l'environnement peut être recréé à l'identique en quelques minutes, qu'il s'agisse d'un environnement de développement, de test ou de production. Cette traçabilité de l'infrastructure est désormais une exigence standard dans les équipes data professionnelles.

La conformité RGPD a été intégrée dès la phase de conception, et non ajoutée a posteriori. Les données sensibles sont supprimées ou pseudonymisées dès la zone **staging** grâce à des procédures de tri automatisées. Une politique de rétention contrôlée, configurée via les règles de lifecycle Azure CLI, assure que les données brutes ne sont pas conservées indéfiniment. Le registre des traitements documente chaque flux de données, sa finalité, sa base légale et les mesures de sécurité associées, conformément aux exigences du RGPD.

En termes de gouvernance et de sécurité, ce projet illustre la mise en pratique du principe du moindre privilège à chaque couche du système : ACL ADLS Gen2 par groupe Azure AD, politiques d'accès Key Vault segmentées par rôle, rôles SQL dédiés par profil, et authentification API par clé. Cette approche multicouche réduit la surface d'attaque et facilite les audits de sécurité. La documentation technique complète, versionnée dans le dépôt Git avec les scripts d'ingestion, les notebooks Databricks et les fichiers Terraform, assure la maintenabilité du système sur le long terme et sa prise en main par une autre équipe sans perte de contexte.

A Script complet – Databricks ETL SIRENE (PySpark)

```

1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import col, substring, current_date,
   when, lit
3 import os
4
5 spark = SparkSession.builder.appName("parquet_etl").getOrCreate()
6
7 # -- Configuration ADLS -----
   -----
8 storage_account = os.getenv("STORAGE_ACCOUNT", "adlsselbrek2025")
9 storage_key     = os.getenv("STORAGE_KEY", "")
10 spark.conf.set(
11     f"fs.azure.account.key.{storage_account}.dfs.core.windows.net",
12     storage_key)
13
14 ADLS_RAW = f"abfss://raw@{storage_account}.dfs.core.windows.net"
15 ADLS_STG = f"abfss://staging@{storage_account}.dfs.core.windows.net"
16
17 # -- Lecture Parquet -----
   -----
18 COLONNES = ["siret", "siren", "denominationUniteLegale",
19             "activitePrincipaleEtablissement", "nomenclatureActivitePrincipaleEtab",
20             "codePostalEtablissement", "libelleCommuneEtablissement",
21             "codeCommuneEtablissement", "etatAdministratifEtablissement",
22             "dateCreationEtablissement", "trancheEffectifsEtablissement",
23             "categorieJuridiqueUniteLegale", "economieSocialeEtablissement",
24             "caractereEmployeurUniteLegale", "dateDebut",
25             "typeVoieEtablissement", "libelleVoieEtablissement",
26             "numeroVoieEtablissement", "complementAdresseEtablissement",
27             "codeCedexEtablissement", "libelleCedexEtablissement"]

```

```

28
29 df_raw = spark.read.parquet(f"{ADLS_RAW}/Parquet/
    StockEtablissement_utf8.parquet")
30 df = df_raw.select([c for c in COLONNES if c in df_raw.columns])
31
32 # -- Filtres -----
    -----
33 df = df.filter(col("etatAdministratifEtablissement") == "A")
34 df = df.filter(substring(col("codeCommuneEtablissement"),1,2)
35                 .isin(["02","59","60","62","80"]))
36
37 # -- Suppression colonnes vides -----
    -----
38 empty_cols = [c for c in df.columns
39                if df.filter(col(c).isNotNull()).count() == 0]
40 df = df.drop(*empty_cols)
41
42 # -- Conformite RGPD : suppression colonnes sensibles -----
    -----
43 SENSIBLES = ["nomUsageUniteLegale","prenomUsuelUniteLegale",
44              "dateNaissancePersonnePhysiqueUniteLegale"]
45 df = df.drop(*[c for c in SENSIBLES if c in df.columns])
46
47 # -- Enrichissement -----
    -----
48 dept_map = {"02":"Aisne","59":"Nord","60":"Oise","62":"Pas-de-
    Calais","80":"Somme"}
49 df = df.withColumn("code_departement",
50                    substring(col("codeCommuneEtablissement"),1,2)
51                    )
52 for code, nom in dept_map.items():
53     df = df.withColumn("nom_departement",
54                        when(col("code_departement")==code, lit(nom))
55                        .otherwise(col("nom_departement") if "nom_departement"
56                                   in df.columns
57                                   else lit(None)))
58
59 df = df.withColumn("region", lit("Hauts-de-France"))
60 df = df.withColumn("date_traitement", current_date())
61
62 # -- Sauvegarde staging -----
    -----

```

```

60 df.coalesce(1).write.mode("overwrite") \
61   .partitionBy("code_departement") \
62   .parquet(f"{ADLS_STG}/etablissements_hdf_actifs/")
63
64 print(f"ETL termine : {df.count()} etablissements actifs en Hauts
    -de-France")

```

Listing 10 – databricks/parquet_etl.py – ETL complet SIRENE

B Script complet – Ingestion API Géo (fetch_communes.py)

```

1 import requests, json, os, argparse, logging
2 from azure.storage.blob import BlobServiceClient
3
4 logging.basicConfig(level=logging.INFO,
5                     format="%(asctime)s %(levelname)s %(message)s
6                     ")
7 logger = logging.getLogger(__name__)
8
9 CONN_STR = os.getenv("AZURE_STORAGE_CONNECTION_STRING", "")
10
11 def fetch_communes(departements, container='raw',
12                   datalake_path='geo', local_output=None):
13     client = BlobServiceClient.from_connection_string(CONN_STR)
14     if CONN_STR else None
15     for dept in departements:
16         url = f"https://geo.api.gouv.fr/departements/{dept}/
17         communes"
18         params = {"fields": "nom,code,codesPostaux,population,
19         surface,contour",
20                 "format": "json", "geometry": "contour"}
21         try:
22             r = requests.get(url, params=params, timeout=10)
23             r.raise_for_status()
24             data = r.json()
25             logger.info(f"Dept {dept}: {len(data)} communes")
26             # Sauvegarde locale
27             if local_output:
28                 path = os.path.join(local_output, f"communes_{
29                 dept}.json")
30                 with open(path, "w", encoding="utf-8") as f:

```

```

26         json.dump({"communes": data}, f, ensure_ascii
= False)
27         # Upload ADLS
28         if client:
29             blob = client.get_blob_client(
30                 container=container,
31                 blob=f"{datalake_path}/communes_{dept}.json")
32             blob.upload_blob(json.dumps({"communes": data},
33                                     ensure_ascii=False), overwrite=
True)
34             logger.info(f"Upload ADLS: {datalake_path}/
communes_{dept}.json")
35             except requests.exceptions.Timeout:
36                 logger.error(f"Timeout pour departement {dept}")
37             except requests.exceptions.HTTPError as e:
38                 logger.error(f"Erreur HTTP {e.response.status_code}
pour {dept}")
39
40 if __name__ == "__main__":
41     parser = argparse.ArgumentParser()
42     parser.add_argument("--departements", nargs="+",
43                         default=["02", "59", "60", "62", "80"])
44     parser.add_argument("--container", default="raw")
45     parser.add_argument("--local-output", default=None)
46     args = parser.parse_args()
47     fetch_communes(args.departements, args.container,
48                   local_output=args.local_output)

```

Listing 11 – ingestion/API/fetch_communes.py

C Script complet – Export SQL vers ADLS (sql_to_adls_bis.py)

```

1 import os, pandas as pd
2 from sqlalchemy import create_engine
3 from azure.storage.blob import BlobServiceClient
4
5 SERVER = os.getenv("AZURE_SQL_SERVER_BIS")
6 DATABASE = os.getenv("AZURE_SQL_DATABASE_BIS")
7 LOGIN = os.getenv("AZURE_SQL_LOGIN_BIS")
8 PASSWORD = os.getenv("AZURE_SQL_PASSWORD_BIS")

```

```
9 CONN_STR = os.getenv("AZURE_STORAGE_CONNECTION_STRING")
10
11 TABLES = ["dim_commune", "stg_population", "
    stg_creation_entreprises",
12             "stg_deces", "stg_emploi_chomage", "stg_fecondite",
13             "stg_filosofi", "stg_logement", "stg_menage", "
    stg_naissances"]
14
15 engine = create_engine(
16     f"mssql+pyodbc://{LOGIN}:{PASSWORD}@{SERVER}"
17     f"/{DATABASE}?driver=ODBC+Driver+18+for+SQL+Server&Encrypt=
    yes")
18
19 blob_service = BlobServiceClient.from_connection_string(CONN_STR)
20
21 for table in TABLES:
22     print(f"Exportation {table}...")
23     df = pd.read_sql(f"SELECT * FROM dbo.{table}", engine)
24     parquet_bytes = df.to_parquet(index=False)
25     blob = blob_service.get_blob_client(
26         container="raw", blob=f"sql-bis/{table}.parquet")
27     blob.upload_blob(parquet_bytes, overwrite=True)
28     print(f"    -> {len(df)} lignes exportees vers raw/sql-bis/{
    table}.parquet")
29
30 print("Export termine.")
```

Listing 12 – analytics/sql_to_adls_bis.py